

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет
информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра Информатики
Дисциплина «Избранные главы информатики»

ОТЧЕТ
к лабораторной работе №4
на тему:
**«РАБОТА С ФАЙЛАМИ, КЛАССАМИ, СЕРИАЛИЗАТОРАМИ,
РЕГУЛЯРНЫМИ ВЫРАЖЕНИЯМИ И СТАНДАРТНЫМИ
БИБЛИОТЕКАМИ»**
БГУИР 6-05-0612-02

Выполнил студент группы 453503
ВОДНЕВ Кирилл Владимирович

(дата, подпись студента)

Проверил доцент каф. Информатики
ЖВАКИНА Анна Васильевна

(дата, подпись преподавателя)

Минск 2026

Задание 1. Исходные данные представляют собой словарь. Необходимо поместить их в файл, используя сериализатор. Организовать считывание данных, поиск, сортировку в соответствии с индивидуальным заданием. Обязательно использовать классы. Реализуйте два варианта: 1) формат файлов CSV; 2) модуль pickle.

Вар-т	Условие
9.	В анкетных данных обозначены фамилия, пол, рост. Определите средний рост женщин, фамилию самого высокого мужчины, есть ли в группе хотя бы два человека одного роста. Выведите информацию о человеке, введенном с клавиатуры

Листинг кода файла person.py для реализации классов пользователей для хранения персональных данных:

```
class PrintableMixin: 2 usages
    """Mixin to print all persons passed as an argument."""

    @staticmethod 1 usage
    def print_all(persons):
        if not persons:
            print("No data available.")
            return

        print("\n" + "=" * 60)
        print("All Persons:")
        print("=" * 60)

        for index, person in enumerate(persons, 1):
            print(f"{index}. {person}")

        print("=" * 60 + "\n")

class Person(PrintableMixin): 25 usages
    """Base class for a person with surname, gender and height."""

    all_persons = [] # static attribute - shared by all instances

    def __init__(self, last_name: str, gender: str, height: float):
        self.last_name = last_name # uses property setter
        self.gender = gender.upper() # dynamic attribute
        self.height = height # dynamic attribute
        Person.all_persons.append(self) # add to static list

    def __str__(self) -> str:
        gender_str = "Female" if self.gender == "F" else "Male"
        return f"{self.last_name:<15} | Gender: {gender_str:<6} | Height: {self.height} cm"

    @property 8 usages (6 dynamic)
    def last_name(self) -> str:
        return self._last_name

    @last_name.setter 7 usages (6 dynamic)
    def last_name(self, name: str):
        self._last_name = name.title()

    @classmethod 1 usage
    def search_by_letter(cls, letter: str) -> list:
        return [person for person in cls.all_persons
                if person.last_name.upper().startswith(letter.upper())]
```

```

@classmethod 1 usage
def search_by_letter(cls, letter: str) -> list:
    return [person for person in cls.all_persons
            if person.last_name.upper().startswith(letter.upper())]

@classmethod 2 usages
def search_by_surname(cls, surname: str):
    for person in cls.all_persons:
        if person.last_name.lower() == surname.lower():
            return person
    return None

@classmethod 1 usage
def get_average_height_women(cls) -> float | None:
    women = [person for person in cls.all_persons if person.gender == "F"]

    if women:
        return sum(person.height for person in women) / len(women)
    return None

@classmethod 1 usage
def get_tallest_man(cls):
    men = [person for person in cls.all_persons if person.gender == "M"]

    if men:
        return max(men, key=lambda person: person.height)
    return None

@classmethod 1 usage
def check_same_height_pair(cls) -> bool:
    heights = [person.height for person in cls.all_persons]

    # Check if any height appears more than once
    return len(heights) != len(set(heights))

@classmethod
def get_people_with_height(cls, height: float) -> list:
    return [person for person in cls.all_persons if person.height == height]

class BusinessPerson(Person, PrintableMixin):
    """Extended person class with business information. Inherits from Person and PrintableMixin."""

    def __init__(self, last_name: str, gender: str, height: float, company: str):
        super().__init__(last_name, gender, height)
        self.company = company

    def __str__(self) -> str:
        gender_str = "Female" if self.gender == "F" else "Male"
        return (f"{self.last_name:<15} | Gender: {gender_str:<6} | "
                f"Height: {self.height} cm | Company: {self.company}")

```

Листинг кода файла `storage_csv.py` для обработки сохранения и загрузки данных о пользователях из CSV-файлов:

```

import csv
import os
from task1.person import Person

def save_to_csv(): 2 usages
    file_path = get_path_to_file()

    try:
        with open(file_path, "w", newline='', encoding='utf-8') as file_csv_out:
            writer = csv.writer(file_csv_out)
            writer.writerow(["Surname", "Gender", "Height"]) # header

```

```

        for person in Person.all_persons:
            writer.writerow([person.last_name, person.gender, person.height])

    print(f"✓ Successfully saved {len(Person.all_persons)} persons to CSV")

except IOError as e:
    print(f"× Error saving to CSV: {e}")

def load_from_csv(): 2 usages
    file_path = get_path_to_file()

    try:
        if not os.path.exists(file_path):
            print(f"× File not found: {file_path}")
            return

        Person.all_persons.clear()

        with open(file_path, newline='', encoding='utf-8') as file_csv_in:
            reader_csv = csv.reader(file_csv_in)
            next(reader_csv) # skip header

            loaded_count = 0
            for row in reader_csv:
                if row and len(row) >= 3:
                    try:
                        surname, gender, height = row[0], row[1], float(row[2])
                        Person(surname, gender, height)
                        loaded_count += 1
                    except ValueError:
                        print(f"× Invalid data format in row: {row}")

            print(f"✓ Successfully loaded {loaded_count} persons from CSV")

    except IOError as e:
        print(f"× Error loading from CSV: {e}")

def get_path_to_file() -> str: 2 usages
    current_file_dir = os.path.dirname(os.path.abspath(__file__))
    return os.path.join(current_file_dir, 'files', 'persons.csv')

```

Листинг кода `storage_pickle.py` для обработки сохранения и загрузки данных пользователей с использованием сериализации `pickle`:

```

import os
import pickle
from task1.person import Person

def save_to_pickle(): 2 usages
    file_path = get_path_to_file()

    try:
        with open(file_path, "wb") as pickle_file_out:
            pickle.dump(obj=Person.all_persons, file=pickle_file_out)

        print(f"✓ Successfully saved {len(Person.all_persons)} persons to Pickle")

    except IOError as e:
        print(f"× Error saving to Pickle: {e}")

def load_from_pickle(): 2 usages
    file_path = get_path_to_file()

```

```

try:
    if not os.path.exists(file_path):
        print(f"× File not found: {file_path}")
        return

    Person.all_persons.clear()

    with open(file_path, "rb") as pickle_file_in:
        loaded = pickle.load(pickle_file_in)
        Person.all_persons.extend(loaded)

    print(f"✓ Successfully loaded {len(Person.all_persons)} persons from Pickle")

except (IOError, pickle.PickleError) as e:
    print(f"× Error loading from Pickle: {e}")

def get_path_to_file() -> str: 2 usages
    current_file_dir = os.path.dirname(os.path.abspath(__file__))
    return os.path.join(current_file_dir, 'files', 'persons.pkl')

```

Результат выполнения работы интерфейса, реализованном в файле task1.py:

```

=====
Person Management Menu
=====
1. Add new person
2. Find by surname
3. Find by first letter
4. Calculate average height of women
5. Find tallest man
6. Check for people with same height
7. Display searched person info
8. Show all persons
9. Save to CSV
10. Load from CSV
11. Save to Pickle
12. Load from Pickle
0. Exit

Choose an option: 1

-----
Add New Person
-----
Enter surname (letters only): Ferdinand
Enter gender (M/F): M
Enter height in cm (positive number): 189
✓ Person added successfully

Choose an option: 2

-----
Find by Surname
-----
Enter surname to search: Ferdinand

✓ Found person:
Ferdinand      | Gender: Male   | Height: 189.0 cm

Choose an option: 3

-----
Find by First Letter
-----
Enter the first letter of surname: F

```

```

✓ Found 1 person(s):

1. Ferdinand      | Gender: Male   | Height: 189.0 cm
Choose an option: 4

-----
Average Height of Women
-----

× No women found in the database

Choose an option: 5

-----
Tallest Man
-----

✓ Tallest man:
  Surname: Ferdinand
  Height: 189.0 cm

Choose an option: 6

-----
Same Height Check
-----

× No two people have the same height

Choose an option: 7

-----
Display Person Information
-----
Enter surname to search: Ferdinand

=====
Person Information
=====
Surname: Ferdinand
Gender:   Male
Height:   189.0 cm
=====

Choose an option: 8

=====
All Persons:
=====
1. Ferdinand      | Gender: Male   | Height: 189.0 cm
=====

Choose an option: 9
✓ Successfully saved 1 persons to CSV

Choose an option: 10
✓ Successfully loaded 1 persons from CSV

Choose an option: 11
✓ Successfully saved 1 persons to Pickle

Choose an option: 12
✓ Successfully loaded 1 persons from Pickle

```

Задание 2. В соответствии с заданием своего варианта составить программу для анализа текста. Считать из исходного файла текст. Используя регулярные выражения получить искомую информацию (см. условие), вывести ее на экран и сохранить в другой файл. Заархивировать файл с результатом с помощью модуля `zipfile` и обеспечить получение информации о файле в архиве.

Также выполнить общее задание – определить и сохранить в файл с результатами:

- количество предложений в тексте;
 - количество предложений в тексте каждого вида отдельно (повествовательные, вопросительные и побудительные);
 - среднюю длину предложения в символах (считаются только слова);
 - среднюю длину слова в тексте в символах;
 - количество смайликов в заданном тексте. Смайликом будем считать последовательность символов, удовлетворяющую условиям:
 - первым символом является либо «;» (точка с запятой) либо «:» (двоеточие) ровно один раз;
 - далее может идти символ «-» (минус) сколько угодно раз (в том числе символ минус может идти ноль раз);
 - в конце обязательно идет некоторое количество (не меньше одной) одинаковых скобок из следующего набора: «(», «)», «[», «]»;
 - внутри смайлика не может встречаться никаких других символов.Например, эта последовательность является смайликом: «;-----[[[[[[[».
- Эти последовательности смайликами не являются: «]», «;--», «:», «;», «(».

Вар-т	Условие
9.	Получить список дат (формат 12-05-2007) Из заданной строки получить список слов, у которых последняя буква гласная, а предпоследняя – согласная. определить количество строчных букв; найти последнее слово, содержащее букву 'i' и его номер; вывести строку, исключив из нее слова, начинающиеся с 'i'

Листинг кода файла analyzer.py для анализа текста с использованием регулярных выражений:

```
import re

class BaseAnalyzer: 4 usages
    """Baseclass for all text processors."""

    def __init__(self, text: str):
        self.text = text

    def process(self):
        raise NotImplementedError("Must be overridden in subclass.")

class SentenceAnalyzer(BaseAnalyzer): 3 usages
    """Analyzes sentence statistics from text."""

    def process(self) -> dict: 2 usages
        sentences = re.findall( pattern: r'^(.!?)+[.!?]', self.text)

        declarative = len(re.findall( pattern: r'^(.!?)+\.', self.text))
        interrogative = len(re.findall( pattern: r'^(.!?)+\?', self.text))
        imperative = len(re.findall( pattern: r'^(.!?)+!', self.text))
```



```

if sentences:
    total_words = sum(len(re.findall( pattern: r'\b\w+\b', sentence)) for sentence in sentences)
    avg_sentence_len = total_words / len(sentences)

    total_chars = sum(
        len(''.join(re.findall( pattern: r'\b\w+\b', sentence)))
        for sentence in sentences
    )
    avg_sentence_chars = total_chars / len(sentences)
else:
    avg_sentence_len = 0
    avg_sentence_chars = 0

return {
    "total": len(sentences),
    "declarative": declarative,
    "interrogative": interrogative,
    "imperative": imperative,
    "avg_sentence_length": round(avg_sentence_len, 2),
    "avg_sentence_chars": round(avg_sentence_chars, 2)
}

```

```

class WordAnalyzer(BaseAnalyzer): 3 usages

```

```

    """Analyzes word statistics from text."""

```

```

def process(self) -> dict: 2 usages

```

```

    words = re.findall( pattern: r'\b\w+\b', self.text)

```

```

    avg_word_len = sum(len(word) for word in words) / len(words) if words else 0

```

```

    vowel_cons_words = [
        word for word in words
        if len(word) >= 2 and
        word[-1].lower() in 'aeiou' and
        word[-2].lower() not in 'aeiou' and
        word[-2].isalpha()
    ]

```

```

    lowercase_count = sum(1 for char in self.text if char.islower())

```

```

    last_i_word = None
    last_i_index = None
    for idx, word in enumerate(words, 1):
        if 'i' in word.lower():
            last_i_word = word
            last_i_index = idx

```

```

    text_without_i = re.sub( pattern: r'\b[iI]\w*\b', repl: '', self.text)
    text_without_i = re.sub( pattern: r'\s+', repl: ' ', text_without_i).strip()

```

```

    dates = re.findall( pattern: r'\b(\d{2}-\d{2}-\d{4})\b', self.text)

```

```

return {
    "word_count": len(words),
    "avg_word_length": round(avg_word_len, 2),
    "words_vowel_ending_consonant_penultimate": vowel_cons_words,
    "lowercase_count": lowercase_count,
    "last_word_with_i": last_i_word,
    "last_word_with_i_index": last_i_index,
    "text_without_i_words": text_without_i,
    "dates": dates
}

```



```

class SmileyAnalyzer(BaseAnalyzer): 3 usages
    """Finds all valid smileys in the text."""

    def process(self) -> dict: 2 usages
        full_smileys = re.findall(pattern: r'[:;]-*[(\[\]])+', self.text)

        valid_smileys = []
        for smiley in full_smileys:
            if len(smiley) >= 2:
                dash_end = smiley.find('-')
                if dash_end == -1:
                    bracket_part = smiley[1:]
                else:
                    dash_end = smiley.rfind('-') + 1
                    bracket_part = smiley[dash_end:]

                if bracket_part and all(b == bracket_part[0] for b in bracket_part):
                    valid_smileys.append(smiley)

        return {
            "smiley_count": len(valid_smileys),
            "smileys": valid_smileys
        }

class DateAnalyzer(BaseAnalyzer): 3 usages
    """Extracts dates from text in format DD-MM-YYYY."""

    def process(self) -> dict: 2 usages
        dates = re.findall(pattern: r'\b(\d{2}-\d{2}-\d{4})\b', self.text)

        return {
            "date_count": len(dates),
            "dates": dates
        }

```

Листинг кода файла archive.py для архивирования результатов и управления ZIP-файлами:

```

import os
import zipfile

def ensure_directory_exists(): 2 usages
    """Create files directory if it doesn't exist."""
    file_path = get_path_to_file()
    directory = os.path.dirname(file_path)

    if not os.path.exists(directory):
        os.makedirs(directory)

def archive_file(filename: str) -> dict: 2 usages
    ensure_directory_exists()
    zip_name = get_path_to_file()

    try:
        with zipfile.ZipFile(zip_name, mode: 'w', zipfile.ZIP_DEFLATED) as zip_archive:
            zip_archive.write(filename, arcname=os.path.basename(filename))

        # Get file info from archive
        with zipfile.ZipFile(zip_name, mode: 'r') as zip_archive:
            info = zip_archive.getinfo(os.path.basename(filename))

```

```

        compression_ratio = (
            (1 - info.compress_size / info.file_size) * 100
            if info.file_size > 0 else 0
        )

        return {
            "filename": info.filename,
            "size": info.file_size,
            "compress_size": info.compress_size,
            "compression_ratio": round(compression_ratio, 2)
        }

    except FileNotFoundError:
        print(f"× File not found: {filename}")
        return None
    except zipfile.BadZipFile:
        print("× Error creating ZIP file")
        return None

def save_results_to_file(filename: str, results: list): 2 usages
    ensure_directory_exists()

    try:
        with open(filename, "w", encoding='utf-8') as file:
            for result in results:
                file.write(f"{result}\n\n")
            print(f"✓ Results saved to: {filename}")
    except IOError as e:
        print(f"× Error saving results: {e}")

def get_path_to_file() -> str: 2 usages
    current_file_dir = os.path.dirname(os.path.abspath(__file__))
    return os.path.join(current_file_dir, 'files', 'result.zip')

```

Результат выполнения работы интерфейса, реализованном в файле task2.py:

```

=====
Text Analyzer Menu
=====
1. Analyze sentences
2. Analyze words (Variant 9 specific)
3. Count smileys
4. Extract dates (DD-MM-YYYY)
5. Archive and compress results
6. Show all analysis results
0. Exit

Choose an option: 1

-----
Sentence Analysis
-----

Total sentences:          16

Sentence types:
  • Declarative (.):      10
  • Interrogative (?):    3
  • Imperative (!):       3

Lengths:
  • Avg sentence (words):  6.38 words
  • Avg sentence (chars):  30.5 characters

```

Choose an option: 2

Word Analysis

General statistics:

- Total words: 102
- Average word length: 4.78 characters

Lowercase letters in text: 454

Extracted dates (DD-MM-YYYY):

- 12-05-2007
- 25-12-2020

Words ending with vowel (penultimate is consonant):

Count: 33

Words: Hello, sample, Are, The, date, date, like, write, nature, promise
... and 23 more

Last word containing 'i':

Word: it

Position: #102

Text without words starting with 'i':

Hello world! This a sample text. Are you ready for analysis? The date 12-05-2007 marks an event. Ano...

Choose an option: 3

Smiley Analysis

Total valid smileys: 12

Valid smileys found:

- :)
- :--(
- ;-]
- :---))))
- :-[[[[
- ;-----[[[[[[[[
- :(
- ;)
- :))
- ;-)
- :-]
- ;---)

Choose an option: 4

Date Extraction (DD-MM-YYYY)

Total dates found: 2

Dates:

- 12-05-2007
- 25-12-2020

Choose an option: 5

Archiving Analysis Results

✓ Results saved to: D:\BSUIR\4th semester\IGI\LR4\task2\files\result.txt

✓ Archive created successfully

Filename: result.txt

Original size: 2404 bytes

Compressed size: 731 bytes

Compression ratio: 69.59%

Задание 3. В соответствии с заданием своего варианта доработать программу из ЛР3, используя класс и обеспечить:

а) определение дополнительных параметров: среднее арифметическое элементов последовательности, медиана, мода, дисперсия, СКО последовательности;

б) с помощью библиотеки `matplotlib` нарисовать графики разных цветов в одной координатной оси:

- график по полученным данным разложения функции в ряд, представленным в таблице,
- график соответствующей функции, представленной с помощью модуля `math`. Обеспечить отображение координатных осей, легенды, текста и аннотации.

x	n	$F(x)$	$Math F(x)$	eps

Здесь x – значение аргумента, $F(x)$ – значение функции, n – количество просуммированных членов ряда, $Math F(x)$ – значение функции, вычисленное с помощью модуля `math`.

в) сохранить графики в файл

Вар-т	Условие
9.	$\arccos x = \frac{\pi}{2} - \arcsin x = \frac{\pi}{2} - \sum_{n=0}^{\infty} \frac{(2n)!}{4^n (n!)^2 (2n+1)} x^{2n+1} = \frac{\pi}{2} - x - \frac{x^3}{6} - \dots, x \leq 1$

Листинг кода файла `plot.py` для построения графиков для визуализации степенных рядов:

```
import os
import matplotlib.pyplot as plt
from matplotlib import ticker
from datetime import datetime

plt.rcParams['text.usetex'] = False
plt.rcParams['font.family'] = 'DejaVu Sans'

def get_path_to_file(file: str) -> str:
    """Get absolute path to file in task3/files directory."""

    current_file_dir = os.path.dirname(os.path.abspath(__file__))
    files_dir = os.path.join(current_file_dir, 'files')

    os.makedirs(files_dir, exist_ok=True)

    return os.path.join(files_dir, file)
```

```

def plot_series_vs_math(fx_values: list, math_fx: float, x: float): 2 usages
    """Plot series approximation vs actual math function."""

    n_values = list(range(1, len(fx_values) + 1))
    math_fx_line = [math_fx for _ in n_values]

    fig, ax = plt.subplots(figsize=(13, 8))

    ax.plot(
        *args: n_values,
        fx_values,
        label=f"Series Approximation (x={x:.4f})",
        color="blue",
        marker='o',
        linewidth=2.5,
        markersize=7,
        alpha=0.75,
        markerfacecolor='lightblue',
        markeredgewidth=2
    )

    ax.plot(
        *args: n_values,
        math_fx_line,
        label=f"Exact arccos({x:.4f})",
        color="red",
        linestyle="--",
        linewidth=3,
        alpha=0.85,
        marker='s',
        markersize=6,
        markerfacecolor='mistyrose'
    )

    title = "Arccos Series Approximation vs Math Function"
    ax.set_title(
        title,
        fontsize=14,
        fontweight='bold',
        pad=20
    )

    ax.set_xlabel(xlabel: "Number of Terms (n)", fontsize=12, fontweight='bold')
    ax.set_ylabel(ylabel: "Function Value F(x)", fontsize=12, fontweight='bold')

    ax.legend(
        fontsize=11,
        loc='best',
        framealpha=0.95,
        shadow=True,
        fancybox=True,
        edgecolor='black',
        frameon=True
    )

    ax.grid(visible: True, alpha=0.3, linestyle='--', linewidth=0.7)

    ax.axhline(y: 0, color='black', linewidth=0.8, alpha=0.5)
    ax.axvline(x: 0, color='black', linewidth=0.8, alpha=0.5)
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

    ax.xaxis.set_major_locator(ticker.MaxNLocator(integer=True))
    ax.yaxis.set_major_formatter(ticker.FormatStrFormatter('%0.4f'))

    last_term_value = fx_values[-1]
    last_n = n_values[-1]

```

```

if len(n_values) > 1:
    offset_n = max(1, last_n // 4)
    offset_y = (max(fx_values) - min(fx_values)) * 0.15 if len(set(fx_values)) > 1 else 0.1
else:
    offset_n = 1
    offset_y = 0.1

ax.annotate(
    text=f"Final: F(x) = {last_term_value:.8f}",
    xy=(last_n, last_term_value),
    xytext=(last_n - offset_n, last_term_value + offset_y),
    arrowprops=dict(
        arrowstyle='->',
        connectionstyle='arc3,rad=0.3',
        color='blue',
        lw=2.5
    ),
    fontsize=10,
    bbox=dict(
        boxstyle='round,pad=0.8',
        facecolor='yellow',
        alpha=0.85,
        edgecolor='darkorange',
        linewidth=2
    ),
    ha='center',
    fontweight='bold'
)

```

```

ax.annotate(
    text=f"Exact: arccos({x:.4f}) = {math_fx:.8f}",
    xy=(last_n, math_fx),
    xytext=(last_n - offset_n, math_fx - offset_y),
    arrowprops=dict(
        arrowstyle='->',
        connectionstyle='arc3,rad=-0.3',
        color='red',
        lw=2.5
    ),
    fontsize=10,
    bbox=dict(
        boxstyle='round,pad=0.8',
        facecolor='lightcyan',
        alpha=0.85,
        edgecolor='darkred',
        linewidth=2
    ),
    ha='center',
    fontweight='bold'
)

error = abs(last_term_value - math_fx)
relative_error = (error / abs(math_fx) * 100) if math_fx != 0 else 0

info_text = (
    f"Absolute Error: {error:.2e}\n"
    f"Relative Error: {relative_error:.6f}%\n"
    f"Terms Used: {len(n_values)}"
)

ax.text(
    x=0.02, y=0.98,
    info_text,
    transform=ax.transAxes,
    fontsize=10,
    verticalalignment='top',
    bbox=dict(

```



```

        boxstyle='round,pad=0.9',
        facecolor='lightyellow',
        alpha=0.92,
        edgecolor='black',
        linewidth=1.5
    ),
    fontweight='bold',
    family='monospace'
)

plt.tight_layout()

timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
filename = f"arccos_plot_{timestamp}.png"
filepath = get_path_to_file(filename)

try:
    plt.savefig(*args: filepath, dpi=300, bbox_inches='tight', facecolor='white')
    print(f"\n✓ Plot saved to: {filepath}")
except Exception as e:
    print(f"\n× Error saving plot: {e}")

plt.show()

```

Листинг кода файла series.py для вычисления степенного ряда для функции arccos:

```

import math
from statistics import mean, median, mode, variance, stdev

class BaseSeries: 1 usage
    """Abstract base class for series calculations."""

    def __init__(self, x: float, eps: float):
        self.x = x
        self.eps = eps
        self.n = 0
        self.fx = 0
        self.math_fx = 0
        self.terms = []

    def calculate(self):
        """Calculate series expansion. Must be implemented by subclass."""
        raise NotImplementedError("Must implement calculate method")

    def get_results(self): 1 usage
        return self.x, self.n, self.fx, self.math_fx, self.eps

class ArccosSeries(BaseSeries): 2 usages
    """Calculator for computing arccos(x) using series expansion."""
    MAX_ITERATION = 500

    def calculate(self): 1 usage
        self.math_fx = math.acos(self.x)

        self.fx = math.pi / 2
        self.terms = [self.fx]
        self.n = 0

        x_squared = self.x * self.x

        term = self.x
        self.fx -= term
        self.terms.append(self.fx)

```

```

for n in range(1, self.MAX_ITERATION):

    coeff = (2 * n) * (2 * n - 1) / (4 * n * n)
    term *= coeff * x_squared

    self.fx -= term
    self.terms.append(self.fx)
    self.n = n

    if abs(term) < self.eps:
        break

def get_statistics(self) -> dict: 1 usage
if len(self.terms) < 2:
    raise Exception(
        f"Not enough data points: {len(self.terms)} < 2. "
        f"Try smaller epsilon value (< 0.1)"
    )

try:
    mode_val = mode(self.terms)
except Exception:
    mode_val = "N/A"

return {
    "Mean": mean(self.terms),
    "Median": median(self.terms),
    "Mode": mode_val,
    "Variance": variance(self.terms),
    "Stdev": stdev(self.terms),
}

```

Результат выполнения работы интерфейса, реализованном в файле task3.py:

```

      Arccos Series Expansion Calculator

Formula: arccos(x) = π/2 - arcsin(x) =
π/2 - Σ [(2n)! / (4^n * (n!)^2 * (2n+1))] * x^(2n+1)

Valid for: |x| ≤ 1

This program:
• Calculates arccos(x) using series expansion
• Compares with math.acos() result
• Computes statistical measures
• Plots comparison graph
• Saves results to file

-----
                        Input Parameters
-----
Enter x (|x| ≤ 1): 0.7
Enter epsilon (0 < eps < 1): 0.0001

Calculating arccos series expansion...

=====
                        CALCULATION RESULTS
=====

--- Main Results ---
+-----+-----+-----+-----+-----+-----+
| x | n | F(x) | arccos(x) | eps | Error |
+-----+-----+-----+-----+-----+-----+
| 0.7000000000 | 10 | 0.5906872556 | 0.7953988302 | 0.0001000000 | 0.2047115746 |
+-----+-----+-----+-----+-----+-----+

```

```

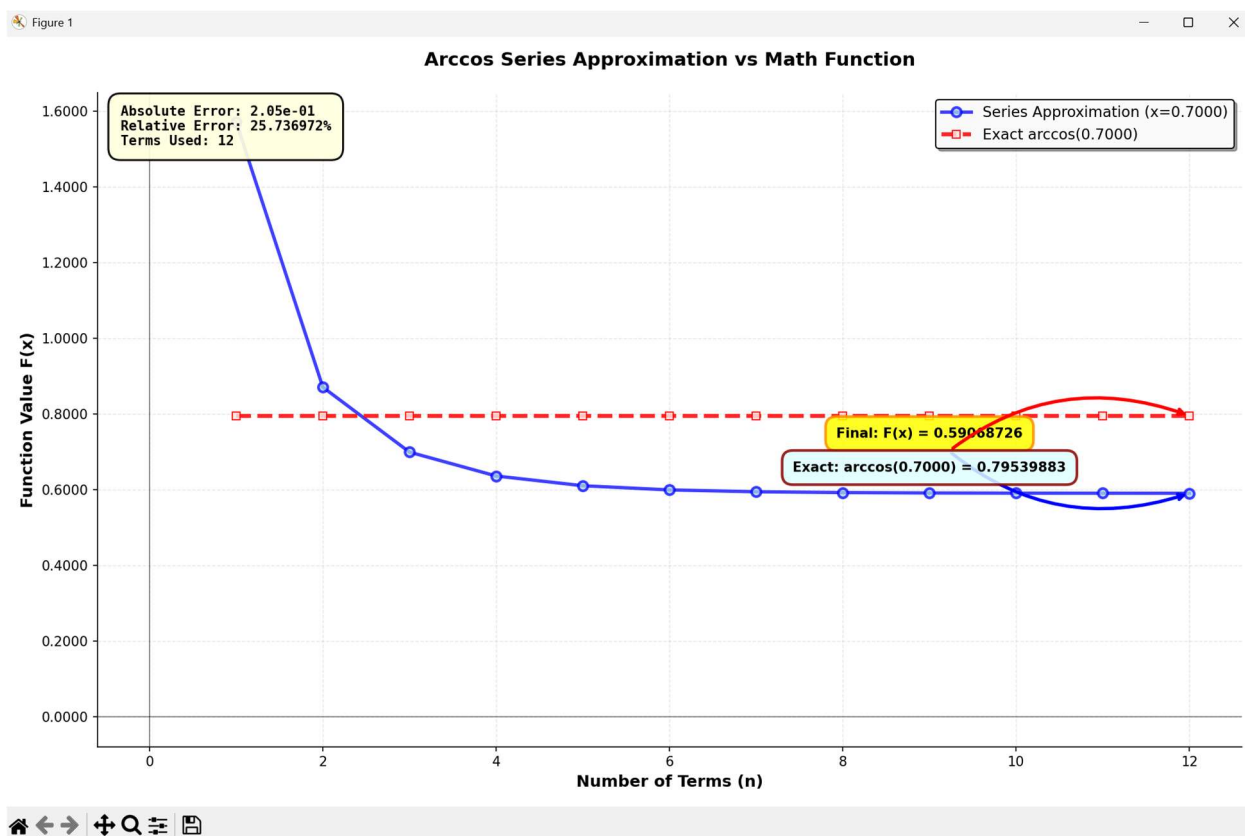
--- Statistical Measures of Series Terms ---
+-----+-----+
| Measure      | Value |
+-----+-----+
| Mean         | 0.711517 |
+-----+-----+
| Median       | 0.597067 |
+-----+-----+
| Mode         | 1.5708 |
+-----+-----+
| Variance     | 0.0798402 |
+-----+-----+
| Standard Deviation | 0.28256 |
+-----+-----+

=====
✓ Convergence achieved in 10 iterations
✓ Absolute error: 2.05e-01
✓ Relative error: 25.736972%
Warning: Error is large. Consider smaller epsilon.

Generating plot...

✓ Plot saved to: D:\BSUIR\4th semester\IGI\LR4\task3\files\arccos_plot_20260204_203615.png

```



Задание 4. В соответствии с заданием своего варианта разработать базовые классы и классы наследники.

Программа должна содержать следующие базовые функции:

- 1) ввод значений параметров пользователем;
- 2) проверка корректности вводимых данных;

3) построение, закрашивание фигуры в выбранный цвет, введенный с клавиатуры, и подпись фигуры текстом, введенным с клавиатуры;

4) вывод фигуры на экран и в файл.

Вар-т	Условие
9.	Построить правильный n-угольник со стороной a.

Листинг кода файла plot.py для построения правильных n-полигонов:

```
import os
import math
import matplotlib.pyplot as plt
from datetime import datetime

def get_path_to_file(file: str) -> str: 2 usages
    """Get the path to save files."""
    current_file_dir = os.path.dirname(os.path.abspath(__file__))
    files_dir = os.path.join(current_file_dir, 'files')

    os.makedirs(files_dir, exist_ok=True)

    return os.path.join(files_dir, file)

def draw_polygon(polygon): 2 usages
    """Draw and display regular polygon using matplotlib."""
    n = polygon.n_sides
    a = polygon.side_length
    color = polygon.color
    label = polygon.label

    circumradius = polygon.get_circumradius()

    vertices = []
    for i in range(n):
        angle = 2 * math.pi * i / n - math.pi / 2 # Start from top
        x = circumradius * math.cos(angle)
        y = circumradius * math.sin(angle)
        vertices.append((x, y))

    vertices.append(vertices[0])

    x_coords = [v[0] for v in vertices]
    y_coords = [v[1] for v in vertices]

    plt.figure(figsize=(10, 10))
    plt.plot(*args: x_coords, y_coords, color=color, linewidth=2, label=f'{polygon.get_name()}')
    plt.fill(*args: x_coords, y_coords, color=color, alpha=0.4)
    plt.scatter(x_coords[:-1], y_coords[:-1], color='red', s=50, zorder=5)
    plt.scatter(x: 0, y: 0, color='black', s=100, marker='x', zorder=5)
    plt.title(label if label else f'{polygon.get_name()} (n={n}, a={a:.2f})', fontsize=14, fontweight='bold')
    plt.xlabel('X')
    plt.ylabel('Y')
    plt.axis('equal')
    plt.grid(visible=True, alpha=0.3)
    plt.legend()

    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    filename = f"polygon_{n}_{timestamp}.png"
    filepath = get_path_to_file(filename)

    plt.savefig(*args: filepath, dpi=150, bbox_inches='tight')
    print(f"\n Plot saved to: {filepath}")

    plt.show()
```

```

def draw_rectangle(rectangle): 2 usages
    """Draw and display rectangle using matplotlib."""
    width = rectangle.width
    height = rectangle.height
    color = rectangle.color
    label = rectangle.label

    x_half = width / 2
    y_half = height / 2

    vertices = [
        (-x_half, -y_half),
        (x_half, -y_half),
        (x_half, y_half),
        (-x_half, y_half),
        (-x_half, -y_half)
    ]

    x_coords = [v[0] for v in vertices]
    y_coords = [v[1] for v in vertices]

    plt.figure(figsize=(10, 8))
    plt.plot(*args: x_coords, y_coords, color=color, linewidth=2, label=f'{rectangle.get_name()}')
    plt.fill(*args: x_coords, y_coords, color=color, alpha=0.4)
    plt.scatter(x_coords[:-1], y_coords[:-1], color='red', s=50, zorder=5)
    plt.scatter(x: 0, y: 0, color='black', s=100, marker='x', zorder=5)
    plt.title(label if label else f'{rectangle.get_name()} ({width:.2f}x{height:.2f})',
              fontsize=14, fontweight='bold')
    plt.xlabel('X')
    plt.ylabel('Y')
    plt.axis('equal')
    plt.grid(visible: True, alpha=0.3)
    plt.legend()

    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    filename = f"rectangle_{timestamp}.png"
    filepath = get_path_to_file(filename)

    plt.savefig(*args: filepath, dpi=150, bbox_inches='tight')
    print(f"\n Plot saved to: {filepath}")

    plt.show()

```

Листинг кода файла shapes.py для реализации базовых классов правильных n-полигонов:

```

from abc import ABC, abstractmethod
import math
from matplotlib.colors import is_color_like

class GeometricFigure(ABC): 2 usages
    """Abstract base class for geometric figures."""

    @abstractmethod
    def area(self) -> float:
        """Calculate area."""
        pass

    @abstractmethod 4 usages (4 dynamic)
    def get_name(self) -> str:
        """Return figure name."""
        pass

```

```

class RegularPolygon(GeometricFigure): 2 usages
    """Regular polygon with n equal sides and angles."""

    def __init__(self, n_sides: int, side_length: float, color: str, label: str = ""):
        """Initialize regular polygon."""
        if n_sides < 3:
            raise ValueError("Number of sides must be >= 3")
        if side_length <= 0:
            raise ValueError("Side length must be > 0")

        self.n_sides = n_sides
        self.side_length = side_length
        self._color = FigureColor(color)
        self.label = label

    def area(self) -> float: 1 usage
        """Area = (n * a^2) / (4 * tan(pi/n))"""
        cot = 1 / math.tan(math.pi / self.n_sides)
        return (self.n_sides * self.side_length ** 2 * cot) / 4

    def get_name(self) -> str: 5 usages (4 dynamic)
        return f"Regular {self.n_sides}-gon"

    def get_perimeter(self) -> float: 1 usage
        """Perimeter = n * a"""
        return self.n_sides * self.side_length

    def get_apothem(self) -> float: 1 usage
        """Apothem = a / (2 * tan(pi/n))"""
        return self.side_length / (2 * math.tan(math.pi / self.n_sides))

    def get_circumradius(self) -> float: 2 usages (1 dynamic)
        """Circumradius = a / (2 * sin(pi/n))"""
        return self.side_length / (2 * math.sin(math.pi / self.n_sides))

```

```

class FigureColor: 2 usages
    """Validates and stores figure color."""

    def __init__(self, color: str):
        self._color = self._validate_color(color)

    @property 7 usages (2 dynamic)
    def color(self) -> str:
        return self._color

    @color.setter 4 usages (4 dynamic)
    def color(self, new_color: str):
        self._color = self._validate_color(new_color)

    @staticmethod 2 usages
    def _validate_color(color: str) -> str:
        if is_color_like(color):
            return color
        print(f"⚠ Warning: '{color}' is not valid. Using 'black'\n")
        return "black"

```

```

class RegularPolygon(GeometricFigure): 2 usages
    """Regular polygon with n equal sides and angles."""

    def __init__(self, n_sides: int, side_length: float, color: str, label: str = ""):
        """Initialize regular polygon."""
        if n_sides < 3:
            raise ValueError("Number of sides must be >= 3")
        if side_length <= 0:
            raise ValueError("Side length must be > 0")

```



```

        self.n_sides = n_sides
        self.side_length = side_length
        self._color = FigureColor(color)
        self.label = label

    def area(self) -> float: 1 usage
        """Area = (n * a^2) / (4 * tan(pi/n))"""
        cot = 1 / math.tan(math.pi / self.n_sides)
        return (self.n_sides * self.side_length ** 2 * cot) / 4

    def get_name(self) -> str: 5 usages (4 dynamic)
        return f"Regular {self.n_sides}-gon"

    def get_perimeter(self) -> float: 1 usage
        """Perimeter = n * a"""
        return self.n_sides * self.side_length

    def get_apothem(self) -> float: 1 usage
        """Apothem = a / (2 * tan(pi/n))"""
        return self.side_length / (2 * math.tan(math.pi / self.n_sides))

    def get_circumradius(self) -> float: 2 usages (1 dynamic)
        """Circumradius = a / (2 * sin(pi/n))"""
        return self.side_length / (2 * math.sin(math.pi / self.n_sides))

    def describe(self) -> str: 1 usage
        """Return polygon description."""
        return (
            "\n" + "=" * 60 + "\n"
            f"Figure: {self.get_name()}\n"
            f"Color: {self._color.color}\n"
            f"Number of sides: {self.n_sides}\n"
            f"Side length: {self.side_length:.4f}\n"
            f"Perimeter: {self.get_perimeter():.4f}\n"
            f"Apothem: {self.get_apothem():.4f}\n"
            f"Circumradius: {self.get_circumradius():.4f}\n"
            f"Area: {self.area():.4f}\n"
            f"Label: {self.label if self.label else 'N/A'}\n"
            + "=" * 60 + "\n"
        )

    @property 3 usages (2 dynamic)
    def color(self) -> str:
        return self._color.color

    @color.setter 2 usages (2 dynamic)
    def color(self, new_color: str):
        self._color.color = new_color

```

```

class Rectangle(GeometricFigure): 2 usages
    """Rectangle with width and height."""

    def __init__(self, width: float, height: float, color: str, label: str = ""):
        """Initialize rectangle."""
        if width <= 0 or height <= 0:
            raise ValueError("Width and height must be > 0")

        self.width = width
        self.height = height
        self._color = FigureColor(color)
        self.label = label

    def area(self) -> float: 1 usage
        """Area = width * height"""
        return self.width * self.height

```



```

def get_name(self) -> str: 5 usages (4 dynamic)
    return "Rectangle"

def get_perimeter(self) -> float: 1 usage
    """Perimeter = 2 * (width + height)"""
    return 2 * (self.width + self.height)

def get_diagonal(self) -> float: 1 usage
    """Diagonal = sqrt(width^2 + height^2)"""
    return math.sqrt(self.width ** 2 + self.height ** 2)

def describe(self) -> str: 1 usage
    """Return rectangle description."""
    return (
        "\n" + "=" * 60 + "\n"
        f"Figure: {self.get_name()}\n"
        f"Color: {self._color.color}\n"
        f"Width: {self.width:.4f}\n"
        f"Height: {self.height:.4f}\n"
        f"Perimeter: {self.get_perimeter():.4f}\n"
        f"Diagonal: {self.get_diagonal():.4f}\n"
        f"Area: {self.area():.4f}\n"
        f"Label: {self.label if self.label else 'N/A'}\n"
        + "=" * 60 + "\n"
    )

@property 3 usages (2 dynamic)
def color(self) -> str:
    return self._color.color

@color.setter 2 usages (2 dynamic)
def color(self, new_color: str):
    self._color.color = new_color

```

Результат выполнения работы интерфейса, реализованном в файле task4.py:

```

Regular Polygon Constructor

Task: Build a regular n-gon with side length a

A regular polygon has:
• All sides of equal length
• All angles equal
•  $n \geq 3$  (triangle minimum)

Key Formulas:
• Area =  $(n * a^2) / (4 * \tan(\pi / n))$ 
• Perimeter =  $n * a$ 
• Apothem =  $a / (2 * \tan(\pi / n))$ 
• Circumradius =  $a / (2 * \sin(\pi / n))$ 

This program:
• Creates regular polygons with user-defined parameters
• Calculates area, perimeter, and radii
• Visualizes the polygon
• Saves visualization to file

=====
POLYGON PARAMETERS
=====

Enter number of sides (n >= 3): 10
Enter side length (a > 0): 4
Enter color (e.g., 'blue', 'red', 'green', 'purple', '#FF5733'): yellow
Enter label for polygon (or press Enter to skip): My pentagon

```

Creating regular 10-gon...

Figure: Regular 10-gon

Color: yellow

Number of sides: 10

Side length: 4.0000

Perimeter: 40.0000

Apothem: 6.1554

Circumradius: 6.4721

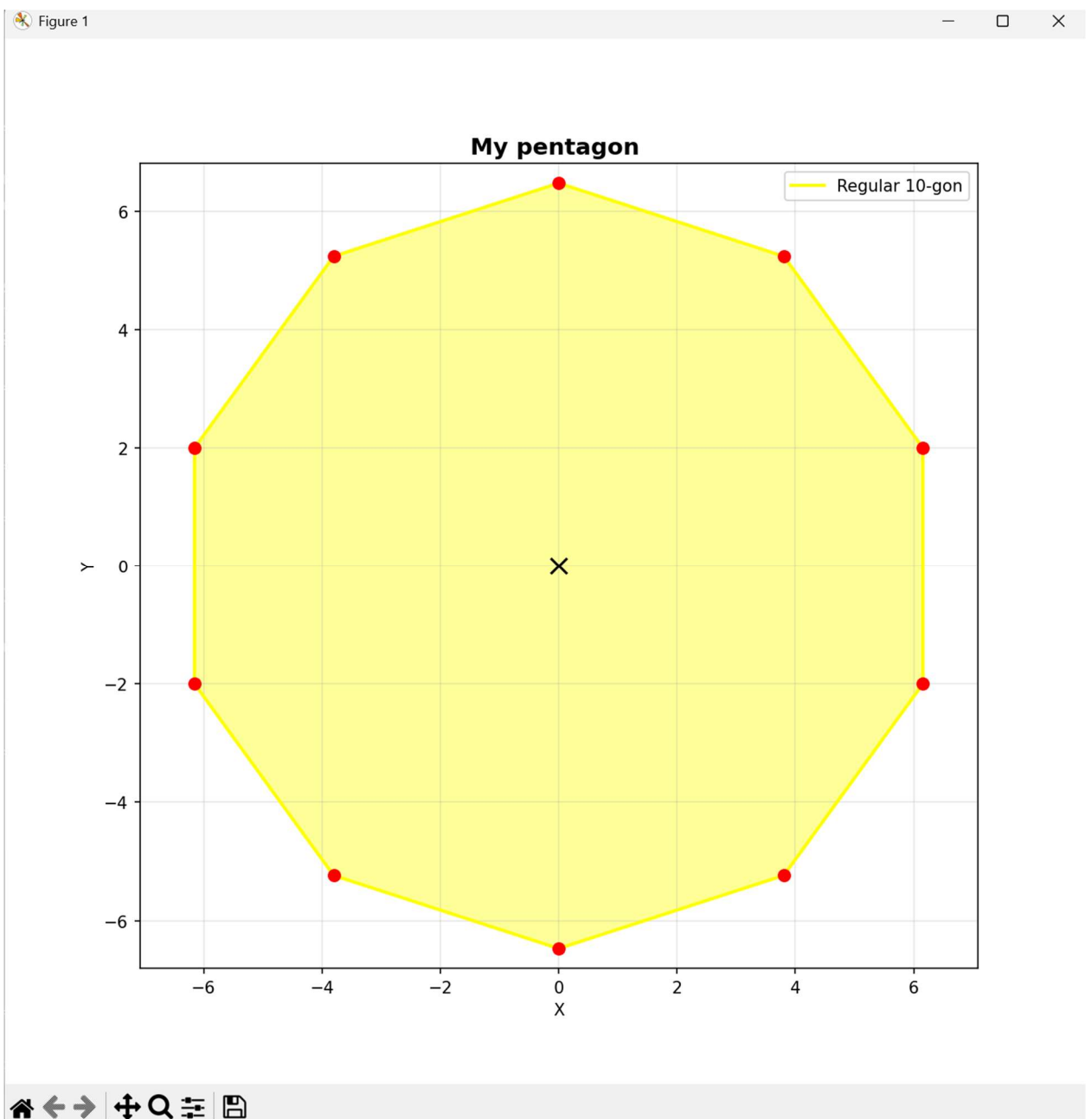
Area: 123.1073

Label: My pentagon

Visualize polygon? (y/n): y

Generating visualization...

✓ Plot saved to: D:\BSUIR\4th semester\I6I\LR4\task4\files\polygon_10_20260204_204030.png



Задание 5. В соответствии с заданием своего варианта исследовать возможности библиотека NumPy при работе с массивами и математическими и статическими операциями. Сформировать целочисленную матрицу $A[n,m]$ с помощью генератора случайных чисел (random).

а) Библиотека NumPy.

1. Создание массива. Функции `array()` и `values()`.
2. Функции создания массива заданного вида.
3. Индексирование массивов NumPy. Индекс и срез.
4. Операции с массивами. Универсальные (поэлементные) функции.

б) Математические и статистические операции.

1. Функция `mean()`
2. Функция `median()`
3. Функция `corrcoef()`
4. Дисперсия `var()`.
5. Стандартное отклонение `std()`

Вар-т	Условие
9.	В матрице поменять местами наибольшие элементы в первом и последнем столбцах. Вычислить значение коэффициента корреляции между элементами первого и последнего столбца. Ответ округлите до сотых.

Листинг кода файла `matrix_creator.py` для создания матриц и статистических операций над ними:

```
import numpy as np
from abc import ABC, abstractmethod

class MatrixBase(ABC):
    """Abstract base class for matrix operations."""
    usage = 1

    def __init__(self, rows: int, cols: int):
        self.rows = rows
        self.cols = cols
        self.matrix = None

    @abstractmethod
    def generate_matrix(self):
        """Generate a matrix."""
        pass

    def get_matrix(self):
        """Return the matrix."""
        usage = 2
        return self.matrix

class MatrixCreator(MatrixBase):
    """Creates integer matrix using random values."""
    usage = 2
```

```

def generate_matrix(self): 1 usage
    """Generate random integer matrix [-100, 100]."""
    self.matrix = np.random.randint(-100, high=101, size=(self.rows, self.cols))

def get_first_column(self): 3 usages
    """Get first column."""
    return self.matrix[:, 0]

def get_last_column(self): 3 usages
    """Get last column."""
    return self.matrix[:, -1]

```

```

def find_max_in_first_col(self): 1 usage
    """Find max element and index in first column."""
    col = self.get_first_column()
    max_val = col.max()
    max_idx = np.argmax(col)
    return max_val, max_idx

```

```

def find_max_in_last_col(self): 1 usage
    """Find max element and index in last column."""
    col = self.get_last_column()
    max_val = col.max()
    max_idx = np.argmax(col)
    return max_val, max_idx

```

```

def swap_max_elements(self): 1 usage
    """Swap maximum elements in first and last columns."""
    max_first, idx_first = self.find_max_in_first_col()
    max_last, idx_last = self.find_max_in_last_col()

    self.matrix[idx_first, 0] = max_last
    self.matrix[idx_last, -1] = max_first

    return (max_first, idx_first), (max_last, idx_last)

```

```

def find_min_elements(self): 1 usage
    """Return min value and its indices."""
    min_val = self.matrix.min()
    indices = np.argwhere(self.matrix == min_val)
    return min_val, indices

```

```

class MatrixAnalyzer: 2 usages
    """Performs statistical operations on matrix."""

```

```

def __init__(self, matrix):
    self.matrix = matrix

```

```

def mean(self): 2 usages (1 dynamic)
    """Mean = sum(elements) / count"""
    return np.mean(self.matrix)

```

```

def median(self): 1 usage
    """Median = middle value when sorted"""
    return np.median(self.matrix)

```

```

def variance(self): 1 usage
    """Variance = mean((x - mean)^2)"""
    return np.var(self.matrix)

```

```

def std_builtin(self): 1 usage
    """Std = sqrt(variance) using NumPy"""
    return round(np.std(self.matrix), 4)

```

```

def std_manual(self): 1 usage
    """Std = sqrt(variance) manual calculation"""
    mean_val = np.mean(self.matrix)
    variance = np.mean((self.matrix - mean_val) ** 2)
    return round(np.sqrt(variance), 4)

@staticmethod 2 usages
def correlation_columns(col1, col2):
    """Calculate correlation coefficient between two columns.

    Returns value rounded to 2 decimal places.
    """
    stacked = np.vstack([col1, col2])
    corr_matrix = np.corrcoef(stacked)
    corr_coef = corr_matrix[0, 1]
    return round(corr_coef, 2)

@property 1 usage
def all_stats(self):
    """Return all statistical measures."""
    return {
        'mean': self.mean(),
        'median': self.median(),
        'variance': self.variance(),
        'std_builtin': self.std_builtin(),
        'std_manual': self.std_manual()
    }

```

Результат выполнения работы интерфейса, реализованном в файле task5.py:

```

NumPy Matrix Operations

Task: Working with integer matrices and statistics

Main operations:
• Generate random integer matrix A[n,m]
• Swap max elements in first and last columns
• Calculate correlation coefficient
• Compute statistical measures

Statistical functions:
• mean()      - Average value
• median()    - Middle value when sorted
• var()       - Variance (dispersion)
• std()       - Standard deviation
• corrcoef()  - Correlation coefficient

=====
MATRIX PARAMETERS
=====

Enter number of rows (n >= 2): 4
Enter number of columns (m >= 2): 5

Generating 4x5 random integer matrix...

-----
GENERATED MATRIX:
-----
[[ 26 -53 -18  5 -30]
 [ 18 -97 -1  38 -9]
 [-76  78 100  49 -98]
 [-16  87  5 -40  79]]

```

```

-----
MAXIMUM ELEMENTS:
-----
Max in first column: 26 at index [0]
Max in last column: 79 at index [3]

-----
MATRIX AFTER SWAP:
-----
[[ 79 -53 -18  5 -30]
 [ 18 -97 -1  38 -9]
 [-76  78 100  49 -98]
 [-16  87  5 -40  26]]

-----
CORRELATION ANALYSIS:
-----
Correlation (before swap): 0.45
Correlation (after swap): 0.46

-----
STATISTICAL MEASURES:
-----
Mean:                2.3500
Median:              2.0000
Variance:            3271.9275
Std Dev (built-in):  57.2008
Std Dev (manual):    57.2008

-----
Minimum value: -98
Found at indices:
[[2 4]]
-----

```

Задание 6. Используя любой из наборов данных (datasets) на ресурсе: <https://www.kaggle.com/datasets> исследовать основные возможности библиотеки Pandas.

а) Библиотека Pandas. Структуры Series и DataFrame

1. Библиотека Pandas. Импорт библиотеки.
2. Структура Series.
3. Создание Series .
4. Функция display.
5. Доступ к элементам Series с использованием .loc или .iloc.
6. Объект DataFrame. Создание.

б) Основные операции

1. Получение информации о датафрейме (по каждому параметру)
2. Индексация и извлечение данных: статистические методы (во сколько раз среднее значение выбранного показателя с максимальным значением другого параметра (например, самый молодой, самый быстрый и т.д.) выше среднего выбранного показателя с минимальным значением)

Листинг кода файла pandas_analyzer.py для анализа классов Pandas Series и DataFrame:

```
import pandas as pd
from IPython.display import display

class PandasBasics: 1 usage
    """Demonstrates basic Pandas Series and DataFrame operations"""

    @staticmethod 1 usage
    def show_series():
        """Creates and displays a Series with custom index"""
        series = pd.Series( data=[100, 200, 300], index=['x', 'y', 'z'])
        print("Series content:")
        display(series)
        print(f"Access by .loc['y']: {series.loc['y']}")
        print(f"Access by .iloc[1]: {series.iloc[1]}")

    @staticmethod 1 usage
    def show_dataframe():
        """Creates and displays a simple DataFrame"""
        df = pd.DataFrame({
            "Product": ["Laptop", "Phone", "Tablet"],
            "Price": [1000, 500, 300]
        })
        print("DataFrame content:")
        display(df)

class DataInfo(PandasBasics): 3 usages
    """Extends PandasBasics to provide DataFrame information"""

    def __init__(self, df: pd.DataFrame):
        """Initialize with a DataFrame"""
        self.df = df

    def dataframe_info(self): 1 usage
        """Displays detailed information about the DataFrame"""
        print("DataFrame Information:")
        self.df.info()

class BaseAnalyzer: 2 usages
    """Base class for data analysis operations"""

    def __init__(self, df: pd.DataFrame):
        """Initialize with a DataFrame copy"""
        self.df = df.copy()

    def preprocess(self, numeric_cols): 3 usages
        """Convert columns to numeric and remove invalid rows"""
        for col in numeric_cols:
            if col in self.df.columns:
                self.df[col] = pd.to_numeric(self.df[col], errors='coerce')
        self.df.dropna(subset=numeric_cols, inplace=True)

    def get_average_by_condition(self, condition_col, target_col, condition='max'): 2 usages
        """Calculate average of target column for extreme value in condition column"""
        extreme_value = self.df[condition_col].max() if condition == 'max' else self.df[condition_col].min()
        return self.df[self.df[condition_col] == extreme_value][target_col].mean()

    def get_ratio(self, condition_col, target_col, numeric_cols):
        """Calculate ratio between max and min condition averages"""
        self.preprocess(numeric_cols)
        max_avg = self.get_average_by_condition(condition_col, target_col, condition='max')
        min_avg = self.get_average_by_condition(condition_col, target_col, condition='min')
        return round(max_avg / min_avg if min_avg != 0 else float('inf'), 2)
```

```

class RatingPriceAnalyzer(BaseAnalyzer): 2 usages
    """Analyzes relationship between Rating and Price"""

    def analyze_ratio(self): 1 usage
        """Calculate price ratio between highest and lowest rated products"""
        self.preprocess(['Rating', 'Price'])
        max_rating_idx = self.df['Rating'].idxmax()
        min_rating_idx = self.df['Rating'].idxmin()
        max_rating_price = self.df.loc[max_rating_idx, 'Price']
        min_rating_price = self.df.loc[min_rating_idx, 'Price']
        ratio = round(max_rating_price / min_rating_price, 2)
        return ratio

class QualityReviewsAnalyzer(BaseAnalyzer): 2 usages
    """Analyzes relationship between Quality Score and Reviews"""

    def analyze_gap(self): 1 usage
        """Calculate review count gap between highest and lowest quality products"""
        self.preprocess(['Quality_Score', 'Reviews'])
        max_quality_idx = self.df['Quality_Score'].idxmax()
        min_quality_idx = self.df['Quality_Score'].idxmin()
        max_quality_reviews = self.df.loc[max_quality_idx, 'Reviews']
        min_quality_reviews = self.df.loc[min_quality_idx, 'Reviews']
        gap = round(max_quality_reviews - min_quality_reviews, 2)
        return gap

```

Результат выполнения работы интерфейса, реализованном в файле task6.py:

```

=====
                        Pandas Data Analysis
=====

PART A: Pandas Series and DataFrame Basics
1. Pandas library import and basic operations
2. Series structure and creation
3. Series element access using .loc and .iloc
4. DataFrame object creation and structure
5. DataFrame information display

PART B: Data Analysis Operations
1. Get DataFrame information for each parameter
2. Statistical methods for data indexing
3. Calculate ratios:
    avg(target) at max(condition) / avg(target) at min(condition)

Example Dataset: Product Sales Data
Columns: Product, Price, Rating, Reviews, Quality Score

=====

TASK 6 MENU
=====

1. Part A: Series & DataFrame Basics
2. Part B: Product Data Analysis
3. Return to Main Menu
Select option (1-3): 1

=====
PART A: PANDAS BASICS
=====
Series content:
x    100
y    200
z    300
dtype: int64

```

```

Access by .loc['y']: 200
Access by .iloc[1]: 200

DataFrame content:
   Product  Price
0  Laptop   1000
1   Phone    500
2  Tablet    300

Select option (1-3): 2

=====
PART B: PRODUCT DATA ANALYSIS
=====

Enter path to dataset CSV: D:\BSUIR\4th semester\IGI\LR4\task6\test_data.csv

Dataset loaded: 10 rows, 5 columns

DataFrame Information:
<class 'pandas.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Product         10 non-null    str
1   Price            10 non-null    int64
2   Rating           10 non-null    float64
3   Reviews          10 non-null    int64
4   Quality_Score    10 non-null    float64
dtypes: float64(2), int64(2), str(1)
memory usage: 532.0 bytes

=====
ANALYSIS RESULTS
=====

Question 1: Rating and Price Relationship
How many times higher is price for high-rated vs low-rated products?
Answer: 22.22x

-----

Question 2: Quality Score and Reviews Relationship
What is the difference in average reviews between
high and low quality products?
Answer: 100 reviews

```

Листинг кода файла inputValidator.py для реализации проверки пользовательских данных:

```

def input_data(description, data_type, min_value=None, max_value=None): 24 usages
    """Prompts the user for input, validates type and value constraints, and returns the valid input"""
    while True:
        try:
            user_input = data_type(input(description))

            if min_value is not None and user_input < min_value:
                raise ValueError(f"The value have to be greater then {min_value}")

            if max_value is not None and user_input > max_value:
                raise ValueError(f"The value have to be less then {max_value}")

            return user_input

        except ValueError as e:
            print(f"Error: {e}. Please enter a valid value.")

```

```
def input_with_validator(description, validator): 7 usages
    while True:
        try:
            user_input = input(description)
            if validator(user_input):
                return user_input
            else:
                raise ValueError
        except ValueError as e:
            print(f'Error: {e}. Please enter a valid value.')
```

Листинг кода главного файла main.py для модульного запуска программы:

```
from task1.task1 import task1
from task2.task2 import task2
from task3.task3 import task3
from task4.task4 import task4
from task5.task5 import task5
from task6.task6 import task6
from utils.inputValidator import input_data

def run_task1(): 1 usage
    """Execute Task 1"""
    task1()

def run_task2(): 1 usage
    """Execute Task 2"""
    task2()

def run_task3(): 1 usage
    """Execute Task 3"""
    task3()

def run_task4(): 1 usage
    """Execute Task 4"""
    task4()

def run_task5(): 1 usage
    """Execute Task 5"""
    task5()

def run_task6(): 1 usage
    """Execute Task 6"""
    task6()

def main() -> None: 1 usage
    """Main function to run the program. Displays menu and executes selected tasks."""
    while True:
        print("\n" + "=" * 60)
        print("Laboratory Work №4 - Menu")
        print("=" * 60)
        print("1. Task 1: Personal Data Management System")
        print("2. Task 2: Text Analysis System")
        print("3. Task 3: Arccos Series Expansion Calculator")
        print("4. Task 4: Regular Polygon Constructor")
        print("5. Task 5: NumPy Matrix Operations")
        print("6. Task 6: Pandas Data Analysis")
        print("0. Exit")
        print("=" * 60)
```

```

select = input_data(description: "Input number of task: ", int, min_value: 0, max_value: 6)

match select:
    case 1:
        run_task1()
    case 2:
        run_task2()
    case 3:
        run_task3()
    case 4:
        run_task4()
    case 5:
        run_task5()
    case 6:
        run_task6()
    case 0:
        print("\n✓ Exiting program. Goodbye!")
        break

if __name__ == "__main__":
    main()

```

Результат выполнения работы программы:

```

=====
Laboratory Work №4 - Menu
=====
1. Task 1: Personal Data Management System
2. Task 2: Text Analysis System
3. Task 3: Arccos Series Expansion Calculator
4. Task 4: Regular Polygon Constructor
5. Task 5: NumPy Matrix Operations
6. Task 6: Pandas Data Analysis
0. Exit
=====
Input number of task: asdsfghjk
Error: invalid literal for int() with base 10: 'asdsfghjk'. Please enter a valid value.
Input number of task: 32
Error: The value have to be less then 6. Please enter a valid value.
Input number of task: 1

```

```

    Personal Data Management System

    This program manages personal information including:
    • Surname, Gender, and Height of individuals

    Capabilities:
    ✓ Add new person's data
    ✓ Calculate average height of women
    ✓ Find tallest man's surname
    ✓ Check for people with same height
    ✓ Display information about searched person
    ✓ Save/Load data from CSV and Pickle formats

```